

STPS: Spatio-Temporal Proactive Scheduling for Spiking Neural Networks on Compute-in-Memory Clusters

Your N. Here
Your Institution

Second Name
Second Institution

Abstract

Spiking Neural Networks (SNNs) deployed on Compute-in-Memory (CIM) clusters present a scheduling problem that does not appear in conventional DNN serving: each model’s network-on-chip (NoC) traffic is both *input-dependent* and *time-varying*, so a tenant that looks idle on average can saturate a card’s NoC during a microsecond-scale spike burst. A formulation that schedules every spike is a time-dependent mixed-integer non-linear program (MINLP) and is intractable at admission time; reactive runtime mitigation only observes congestion once the network has already saturated.

This paper makes three contributions. First, it introduces a compact SNN workload fingerprint: an offline profiler reduces each calibration trace to a traffic timeline $E^{(t)}$, burstiness β , mean active components $\bar{K}$, and a recorded hotspot indicator, so admission decisions no longer inspect the full spike trace. Second, it presents **STPS**, a two-stage admission scheduler that uses the fingerprint to choose a card and then search a bounded start offset that places the task’s traffic into a valley of the selected card’s forecast. Third, it evaluates STPS against four fingerprint-blind baselines across Poisson, bursty, and mixed arrivals at 4-card and 16-card scales. STPS is the only production-style policy that lowers NoC congestion in all six arrival/scale cells, reducing congestion ratio by 5–7% and mean queue wait by 7–9% at a 1–2% throughput cost. At 16 cards, STPS keeps the worst-to-best card load ratio below 13.5 $\times$, while baselines reach up to 24 $\times$; its cost is concentrated in the tail, adding 6–12 ticks at $p99$.

1 Introduction

Energy-efficient deep learning has rekindled interest in Spiking Neural Networks (SNNs) on neuromorphic Compute-in-Memory hardware. Production-class platforms now span academic and industrial designs — TrueNorth [10], Loihi and Loihi-2 [1, 13], SpiNNaker-2 [9], and Darwin3 [8] — and their event-driven execution model converts a model’s idle neurons into idle silicon. The promise is attractive enough

that a *Neuromorphic Computing-as-a-Service* cluster — many tenant SNNs sharing one bank of CIM cards — is a plausible deployment target for the next generation of neuromorphic systems.

Scheduling on such a cluster looks superficially like a DNN serving problem and is fundamentally different. A DNN model has a deterministic compute graph: at admission, a placement decision based on FLOPs, memory, and bandwidth is correct for the entire run, and predictable execution is the design goal of recent serving stacks such as Clockwork [5]. An SNN’s per-tick activity, by contrast, is a function of the input. A vision classifier on a quiescent frame fires a handful of spikes; the same model on a high-contrast frame can inject orders of magnitude more traffic into the NoC. Even within one inference, spikes arrive in narrow temporal bursts driven by neural dynamics rather than uniformly across the inference window. Two consequences follow. First, capacity-based placement — best-fit on free cores, dominant-resource fairness on weighted resources [3], power-of-two-choices on instantaneous load [11] — leaves cards that are spatially balanced and temporally hot, because the bursts of co-located tenants align and the per-card NoC saturates. Second, treating each spike as a scheduling unit is infeasible: at admission the scheduler does not yet know which spikes will fire, and even if it did the resulting MINLP would not solve in the per-task latency budget of a serving system.

A pragmatic scheduler must therefore reason about *distributions* of spike activity, not individual spikes. The information that distinguishes a “quiet” task from a bursty one is statistical, available before the task arrives, and small enough to consult in constant time, independent of the trace length. We make this concrete: a Discrete-Time Dynamic Graph (DTDG) view of an SNN trace yields four compact fingerprints — the traffic timeline $E^{(t)}$, global burstiness β , mean active components $\bar{K}$, and a per-population hotspot indicator — of which the first three suffice to drive admission-time placement and timing decisions without re-examining the underlying weight tensor (the hotspot indicator is recorded for a future spatial-splitting extension).

The resulting scheduler, **STPS** (Spatio-Temporal Proactive Scheduling), takes these fingerprints as input. A macro-dispatch stage scores each card with a closed-form weighted sum of fragmentation and burstiness pressure, picking the card whose fragmentation matches the task’s topology and whose forecast burst-density tolerates the task’s β . A phase-shift stage then searches at most $D_{\max}$ start offsets and commits the offset whose addition to the card’s forecast minimises the peak ($O(D_{\max} \cdot H)$). The two stages address orthogonal failure modes — spatial fragmentation and temporal collision — and are the minimum mechanism that closes the gap between capacity-based DNN scheduling and SNN execution dynamics.

Contributions.

- A *workload fingerprint* that distils an SNN’s spatio-temporal injection profile into four scalars/vectors derivable from a calibration trace, sufficient to support admission-time decisions whose cost is constant in the model size and independent of the DTDG length (Section 5.2).
- A two-stage online scheduler that consumes the fingerprint to dispatch tasks across cards and to shift their start offsets, with both stages reducing to closed-form scoring or a bounded search over $D_{\max}$ offsets (Sections 5.3.1 and 5.3.2).
- An evaluation on real and synthetic SNN traces showing that STPS lowers NoC congestion by 5–7 % and queue wait by 7–9 % across arrival modes, holds the 16-card worst-card tail load under $13.5\times$ versus baselines’ $14\text{--}24\times$, isolates the contribution of each stage, and explicitly characterises the long-tail delay cost the scheduler pays (Section 7).

The rest of the paper is organised as follows. Section 2 explains why both capacity-based and per-spike scheduling fail on SNN/CIM clusters; Section 3 positions STPS against prior work; Section 4 fixes notation; Section 5 presents the offline fingerprint and the two online stages; Section 6 describes the prototype; Section 7 reports the RQ1–RQ4 results and the long-tail trade; Section 8 concludes.

2 Background and Motivation

This section establishes what makes SNN serving on CIM clusters distinct from DNN serving, and why two natural design points — capacity-based scheduling and per-spike optimisation — fail. The argument frames the design constraint that drives Section 5: the scheduler needs a workload abstraction that is statistical, small, and consultable in $O(1)$.

2.1 SNN Execution on CIM Clusters

A CIM card hosts a 2D mesh of cores; each core stores a fixed-size slice of synaptic weights in-memory and routes spikes to its neighbours over an on-chip network. This memory-co-located organisation is shared across the production neuromorphic substrates we target — Loihi/Loihi-2 [1, 13], TrueNorth [10], SpiNNaker-2 [9], and Darwin3 [8] — and dictates three properties that shape the scheduling problem.

(i) *Bound state*. A neural *population* is mapped onto one or more cores at compile time. When any pre-synaptic neuron of population j fires, the post-synaptic membrane updates run locally and out-going spikes are injected into the NoC towards population j ’s downstream targets. Once the populations of a task are pinned, the task cannot migrate mid-inference without a costly state copy [17], so admission-time decisions are largely irrevocable.

(ii) *NoC-dominated contention*. The cluster aggregates M such cards; an inter-card interconnect is treated as a separate bandwidth pool. The dominant shared resource is the per-card NoC: when instantaneous injection exceeds its bandwidth ceiling, downstream spikes are queued and tail latency grows.

(iii) *Capacity is the wrong yardstick*. A card’s free-core count is a poor proxy for its real load, since spike injection is sparse and time-varying. Two cards with identical free-core counts can differ by an order of magnitude in their actual NoC pressure once their resident tasks burst together.

2.2 Why Capacity-Based Scheduling Fails

Schedulers from the DNN serving literature place tasks based on scalar resource demands: best-fit or first-fit on heterogeneous resource vectors [14], dominant-resource fairness on a weighted vector [3], power-of-two-choices on instantaneous load [11]. All three are blind to the time dimension. Two tasks that look identical to DRF can have completely different traffic shapes; if the scheduler co-locates a steady-flat tenant and a bursty tenant whose peaks happen to align, the card saturates even though its mean utilisation is well below capacity. We confirm this empirically in Section 7: under bursty arrivals on a 16-card cluster, capacity-based baselines develop a worst-to-best card load ratio above $20\times$, while their mean utilisation stays under 70 %. Migration-based fixes proposed for GPU clusters [16, 17] can rebalance after the fact, but on CIM hardware the population state is large and the migration cost dwarfs the imbalance window in which the move would have helped.

2.3 Why Schedule-Every-Spike Fails

The opposite extreme — treating each spike as a scheduling decision — is computationally infeasible. Even at modest activity levels (a thousand spikes per task per tick, hundreds of co-resident tasks), a per-spike formulation produces a time-dependent MINLP whose state space exceeds the per-task

admission budget by several orders of magnitude. Equally fatal, the scheduler does not know which spikes will fire at admission: the firing pattern depends on the input. A heuristic that schedules at this granularity must either delay admission until the firing trace is observed (eliminating any chance of proactive placement) or guess (eliminating the precision that motivated the per-spike view in the first place).

2.4 The Missing Middle

The scheduler we need lives between these extremes: it must reason about each task’s spike activity as a *distribution* that is known at admission time, small enough to evaluate in $O(1)$, and rich enough to expose both spatial topology (which cards’ fragmentation matches the task) and temporal shape (where in the card’s forecast a new task fits). The next two sections describe how STPS realises this middle: Section 5.2 reduces an SNN’s DTDG to four such quantities, and Sections 5.3.1–5.3.2 consume them in closed form.

3 Related Work

Cluster scheduling for DNN serving. Most production cluster schedulers approach placement as bin-packing under multi-dimensional capacity constraints. Best-fit and first-fit on heterogeneous resource vectors [7, 14], dominant-resource fairness [3], and multi-resource generalisations [4] pick a card whose free vector dominates the task’s request. These policies are blind to temporal load shape and constitute our baselines in Section 7. Schedulers specialised for DNN training (Gandiva [17], Synergy [11], heterogeneity-aware placement [12]) exploit job-level signals — iteration time, GPU sharing, accelerator heterogeneity — but their unit of work is a deterministic GPU kernel whose footprint does not vary with the input. Predictable DNN serving systems such as Clockwork [5] go further still and make execution time itself a constant by centralising scheduling decisions; this is feasible because the underlying kernel cost is data-independent. None of these systems surfaces a per-tick traffic profile to the scheduler, which is the abstraction STPS relies on, because their workload model does not have one.

Cluster-level admission control. A complementary line of work tackles bursty *arrivals* by predicting future load and provisioning ahead. Workload prediction in serving systems [16] estimates request-rate spikes and pre-scales replicas; admission-control systems for cloud quotas [15] reason about queue depths and fairness across tenants. Both predict the *number* of jobs, not the temporal shape of any single job’s hardware demand. The phase-shift stage of STPS is closer to packing in the temporal dimension: rather than predicting future arrivals, it consumes a per-task forecast and chooses a start offset that avoids collision with the card’s existing schedule. The closest classical analogue is gang scheduling on

parallel machines, but gang scheduling coordinates start times *across cards* to enable communication; STPS coordinates start times *on a single card* to spread NoC pressure.

Neuromorphic mapping and validation. Neuromorphic compilers for production substrates (TrueNorth [10], Loihi [1, 13], SpiNNaker-2 [9], Darwin3 [8]) focus on a static problem: how to partition and place a single model’s populations onto cores so that on-chip routing meets latency and fan-out constraints. Validation work on SpiNNaker [7] characterises spike-traffic distributions at this single-model granularity. These tools answer “how does one model fit one chip”, which is orthogonal to the multi-tenant placement and timing decisions STPS makes at admission. We treat compiled per-model placements as inputs — the per-population centrality and connected-component structure extracted from the DTDG presupposes that this static mapping has already happened — and ask the next-level question of how to dispatch many such pre-compiled models across many cards.

Fingerprint-driven scheduling. Compressing a workload to a small set of summary statistics is a recurring idea in cluster scheduling and load balancing. STPS’s contribution is the choice of fingerprint: four quantities that together capture both the spatial topology ($\bar{K}$, $\mathbf{c}_{\max}^*$) and temporal shape ($E^{(t)}$, β) of an SNN’s spike traffic, in a representation small enough to drive admission-time decisions and physical enough to derive from a calibration trace — specifically, from SpikingJelly [2] runs of architectures such as Spikformer [18] — without bespoke instrumentation.

4 Preliminaries

This section fixes the notation used by the rest of the paper. Symbols defined here recur in Section 5 (system) and Section 7 (evaluation); we do not re-introduce them later.

4.1 Workload Model

A submitted task τ is a pre-compiled SNN already partitioned by the vendor toolchain into a set $\mathcal{V}_\tau$ of hardware-aligned populations, each population fitting in one CIM core. An *inference* runs for T discrete ticks. At each tick t , only a subset of populations fires; the resulting per-tick weighted graph $G_\tau^{(t)} = (\mathcal{V}_\tau, \mathcal{E}_\tau^{(t)}, W_\tau^{(t)})$ records the active inter-population edges and the expected spike traffic $W_\tau^{(t)}[i, j]$ flowing from population i to population j . The sequence $\mathcal{G}_\tau = \{G_\tau^{(1)}, \dots, G_\tau^{(T)}\}$ is a Discrete-Time Dynamic Graph (DTDG); expectations are taken over a calibration set drawn from the model’s evaluation distribution, so $\mathcal{G}_\tau$ depends on the model, not on any specific input.

We never use $\mathcal{G}_\tau$ directly at scheduling time. Instead, Section 5.2 reduces it to four *fingerprint* quantities: the traffic

timeline $E_\tau^{(t)} \in \mathbb{R}^T$, global burstiness β_τ , mean active components $\bar{K}_\tau$, and per-population hotspot indicator $\mathbf{c}_{\max, \tau}^* \in \mathbb{R}^{|\mathcal{V}|}$. The scheduler reads only these.

4.2 Cluster Model

The cluster is a set of M homogeneous cards indexed by $m \in \{1, \dots, M\}$. Each card holds C CIM cores arranged in a 2D mesh and a NoC with per-card injection ceiling $BW_{\max}$ (spikes/tick). For card m we track its free-core occupancy and, derived from it, the largest contiguous free-block ratio $\rho_m \in [0, 1]$ used by Stage 1 dispatch. The card also maintains a per-tick injection forecast $E_m^{(t)} \in \mathbb{R}^H$ over a sliding horizon H , accumulated from all currently-scheduled tasks' offsetted timelines. When the instantaneous demand on card m exceeds $BW_{\max}$, the excess is queued in a pending-traffic buffer rather than dropped; the engine drains it at the cap rate.

Inter-card transfers are modelled as a separate resource pool but never used by our scheduler, which enforces single-card placement: each task lives entirely on one card for the duration of its inference.

4.3 Scheduling Problem

Tasks arrive online according to an arrival process. On each arrival of τ , the scheduler must (a) commit a card $m^*(\tau) \in \{1, \dots, M\}$ for placement and (b) commit a start offset $\Delta t^*(\tau) \in \{0, 1, \dots, D_{\max}\}$ after which τ 's timeline begins injecting into $E_{m^*}^{(t)}$. Both decisions are irrevocable for the duration of the inference: re-placement requires copying the population state and is prohibitive on CIM hardware.

A scheduler is admissible if, for every τ , it returns $(m^*, \Delta t^*)$ within a per-task budget that is constant in the model size and independent of the DTDG length: it may scan the M cards and a bounded offset window, but never re-reads the underlying weight tensor or spike trace. The system-level objective is to minimise NoC contention — specifically, the fraction of demand pushed back by $BW_{\max}$ and the mean wait in the pending queue — subject to maintaining cluster throughput and bounded tail load imbalance across cards. Section 5 describes the policy STPS uses to make these two decisions; Section 7 measures it against the objectives above.

5 System Design

5.1 Overview

Scheduling SNN inference on a multi-card CIM cluster is hard because spike traffic is both *input-dependent* and *time-varying*: the same model exhibits different injection patterns across inputs and across ticks within one inference. A formulation that schedules every spike exactly is a time-dependent MINLP and is intractable at admission time; reactive runtime mitigation observes congestion only after the NoC has already

saturated. STPS resolves this tension by separating what can be known offline from what must be decided online.

Framework. Figure 1 shows the two-phase pipeline. *Offline*, a profiler runs once per model: it traces the SNN over a calibration set, lifts the trace into a Discrete-Time Dynamic Graph (DTDG), and reduces the DTDG to a four-quantity *fingerprint* stored as a small memory-mapped artefact. *Online*, on each task arrival the scheduler reads only the fingerprint and runs two stages in sequence: macro-card dispatch picks a card m^* , then phase-shift picks a start offset Δt^* . The pair $(m^*, \Delta t^*)$ is committed to the chosen card's forecast and returned to the runtime. Stage 1 scores the M candidate cards in $O(M)$ and Stage 2 scans the $D_{\max} + 1$ offsets over the H -tick horizon in $O(D_{\max} \cdot H)$; both are constant in the model size and independent of the DTDG length, so admission cost does not grow with model complexity.

Stage 1 — Macro-Card Dispatch. The first stage chooses *where* the task lands. For each card m with sufficient free capacity, it computes a single scalar score $S_m(\tau)$ that combines two pressures: (i) how well the task's topological cohesion $\bar{K}_\tau$ matches the card's largest contiguous free-block ratio ρ_m , and (ii) whether the card already hosts bursty workloads when τ is itself bursty ($\beta_\tau > \beta_{hi}$). The card minimising S_m wins. The score is a weighted sum rather than a lexicographic rule, so it degrades smoothly between empty and saturated regimes without policy switching.

Stage 2 — Temporal Phase-Shifting. The second stage chooses *when* the task starts. Spatial dispatch alone does not prevent two bursty tasks from peaking on the same tick; Stage 2 exploits the fact that the task's traffic timeline $E_\tau^{(t)}$ is known at admission. It scans the $D_{\max} + 1$ candidate offsets $\Delta t \in \{0, \dots, D_{\max}\}$ and commits the one whose superposition $E_{m^*}^{(t)} + E_\tau^{(t-\Delta t)}$ has the smallest peak under the bandwidth ceiling $BW_{\max}$. The chosen offset is added back to $E_{m^*}^{(t)}$, updating the forecast that the next admission will see.

The rest of this section justifies the fingerprint design (Section 5.2) and details each online stage (Sections 5.3.1 and 5.3.2); the hotspot indicator and discussion follow in Sections 5.4 and 5.5.

5.2 Workload Fingerprint

We model an SNN inference as a Discrete-Time Dynamic Graph (DTDG) $\mathcal{G} = \{G^{(1)}, \dots, G^{(T)}\}$, where each snapshot $G^{(t)} = (\mathcal{V}, \mathcal{E}^{(t)}, W^{(t)})$ shares the same vertex set $\mathcal{V}$ of hardware-aligned neural populations (each population fits within one CIM core) and a per-tick weight tensor $W_{ij}^{(t)}$ measuring the expected spike traffic on edge $i \rightarrow j$. The expectation is taken over a calibration set drawn from the model's

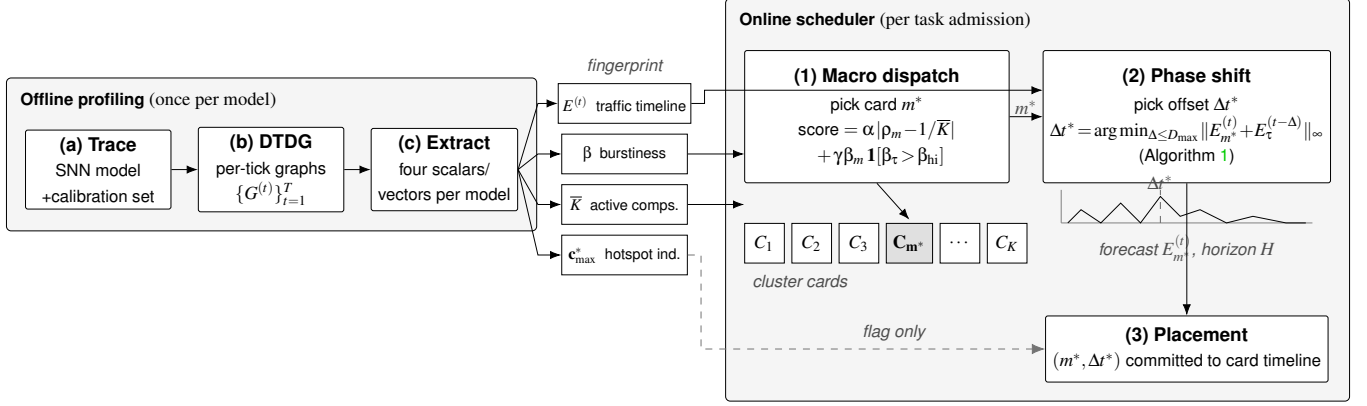


Figure 1: STPS pipeline. *Offline*: a model is traced, lifted into a discrete-time dynamic graph, and reduced to four fingerprints. *Online*: each admission runs in two stages — macro-dispatch picks a card m^* from $\{C_1, \dots, C_K\}$ using $\bar{K}$ and β , then phase-shift picks a start offset $\Delta t^* \leq D_{\max}$ that fits the task’s traffic timeline into a valley of the card’s forecast. The hotspot indicator $\mathbf{c}_{\max}^*$ is recorded but does not drive placement in the current system (dashed arrow).

evaluation distribution, so the fingerprint is independent of any single input.

From this trace we extract three quantities that the online scheduler consumes:

- **Traffic timeline** $E^{(t)} \in \mathbb{R}^T$. The total spike traffic at tick t , averaged over the calibration set. $E^{(t)}$ is the only fingerprint that retains temporal shape.
- **Global burstiness** $\beta = \max_t E^{(t)} / \bar{E}$. Peak-to-mean ratio of the traffic timeline; $\beta \approx 1$ for steady workloads, $\beta \gg 1$ for event-driven bursts.
- **Mean active components** $\bar{K}$. Mean number of connected components in the active-edge subgraph of $G^{(t)}$, averaged over t . $\bar{K} \rightarrow 1$ indicates a topologically cohesive model; $\bar{K} \gg 1$ indicates several concurrent, weakly-coupled subflows.

A fourth quantity, the per-population maximum in-eigenvector centrality $\mathbf{c}_{\max}^*[i] = \max_i c_i^{(t)}$, is recorded as a hotspot indicator and discussed in Section 5.4.

The fingerprint is on the order of tens of kilobytes per model and is computed once at deployment time; subsequent scheduling decisions read it as a memory-mapped artefact.

5.3 Online Scheduler

This subsection details the two stages summarised in Section 5.1.

5.3.1 Stage 1: Macro-Card Dispatch

The cluster constraint we enforce is single-card placement: each model resides entirely on one card. The dispatch problem

therefore reduces to selecting one card among those with sufficient free cores and memory for τ . We score each feasible card m by

$$S_m(\tau) = \alpha \left| \rho_m - \frac{1}{\bar{K}_\tau} \right| + \gamma \beta_m \cdot \mathbb{1}[\beta_\tau > \beta_{\text{hi}}], \quad (1)$$

where ρ_m is the largest contiguous free-block ratio on card m and β_m is its running mean burstiness. The first term matches the task’s topological cohesion against the card’s available shape: cohesive tasks ($\bar{K}_\tau \rightarrow 1$) are drawn to cards with large contiguous blocks ($\rho_m \rightarrow 1$), while decoupled tasks ($\bar{K}_\tau \gg 1$) absorb fragmented cards without penalty. The second term keeps high-burstiness tasks away from cards that already host bursty workloads; it activates only above a threshold β_{hi} so that flat traffic incurs no temporal penalty. The card with minimum S_m wins.

5.3.2 Stage 2: Temporal Phase-Shifting

Cards co-locate multiple tasks, and their traffic timelines superpose on the same on-chip network. Spatial dispatch alone does not prevent two bursty tasks from peaking on the same tick. Stage 2 exploits the fact that the timeline $E_\tau^{(t)}$ is known at admission to delay τ ’s injection so that its peaks fall into the valleys of the existing card-level forecast.

Let $E_m^{(t)}$ be the forecasted background traffic on the selected card m and $D_{\max}$ the deadline-bounded shift budget. The optimal offset is

$$\Delta t^* = \arg \min_{\Delta t \in [0, D_{\max}]} \max_t (E_m^{(t)} + E_\tau^{(t-\Delta t)}), \quad (2)$$

subject to the resulting peak staying below the per-card bandwidth ceiling $BW_{\max}$. When no offset in $[0, D_{\max}]$ satisfies the ceiling, the task is admitted unshifted (offset 0) — it is never

Algorithm 1 Phase-Shift Selection

Require: Background E_m , task E_τ , budget $D_{\max}$, ceiling $BW_{\max}$

Ensure: Offset $\Delta t^* \in [0, D_{\max}]$ (offset 0 if none fits under $BW_{\max}$)

```
1:  $p^* \leftarrow \infty$ ,  $\Delta t^* \leftarrow 0$  ▷ default: admit unshifted
2: for  $\Delta t = 0$  to  $D_{\max}$  do
3:    $p \leftarrow \max_t (E_m^{(t)} + \text{Shift}(E_\tau, \Delta t)^{(t)})$ 
4:   if  $p \leq BW_{\max}$  and  $p < p^*$  then
5:      $p^* \leftarrow p$ ,  $\Delta t^* \leftarrow \Delta t$ 
6:   end if
7: end for
8:  $E_m \leftarrow E_m + \text{Shift}(E_\tau, \Delta t^*)$ 
9: return  $\Delta t^*$ 
```

dropped — and simply contributes its share to congestion; we call this an *offset-infeasible* admission, and it is rare except at extreme load. Algorithm 1 expresses this as a single linear scan over $D_{\max} + 1$ candidate offsets; the cost is $O(D_{\max} \cdot H)$ per task.

5.4 Spatial Mapping and Hotspot Indicator

Once a card and offset are fixed, populations are mapped to PIM cores by the vendor compiler under standard hardware constraints. The fingerprint exposes $\mathbf{c}_{\max}^*$ as a per-population indicator of compute concentration: a population whose centrality exceeds a threshold θ_c is a candidate for splitting across adjacent cores, trading a small spatial footprint for reduced peak SOP pressure. In the current system this flag is recorded on the task but does not drive placement; we leave the centrality-driven splitting pass, and its end-to-end evaluation, to future work.

5.5 Discussion

Two design choices warrant comment. First, all four fingerprint quantities are scalars or short vectors; the scheduler never inspects the underlying $T \times |\mathcal{V}|^2$ trace at runtime. This keeps admission cost independent of model size and is what allows STPS to interleave with arrival rates of thousands of tasks per second. Second, the macro-dispatch score in Equation 1 is deliberately a weighted sum rather than a lexicographic policy: the same scalar handles both empty and saturated regimes, so STPS degrades gracefully as cluster load increases rather than switching between policies.

6 Implementation

We implement STPS as a Python prototype on top of a discrete-event simulator of a multi-card CIM cluster. The system is partitioned into three packages so that the offline profiler, the online scheduler, and the simulation har-

ness can evolve independently. The fingerprint extractor (FINGERPRINT/, ~ 2 kLoC) lifts an SpikingJelly [2] run into a DTDG and reduces it to the four quantities of Section 5.2; the resulting artefact is persisted as a memory-mapped .npz blob ($\sim$ tens of kilobytes per model) and re-loaded lazily at admission time, so cold-start cost is a single file read. The online scheduler (SCHEDULE/, ~ 0.8 kLoC) implements both stages and a pluggable registry that exposes RR, BestFit, DRF, P2C, and the STPS variants behind a uniform interface; the registry also hosts the stage ablations as named entry points so the evaluation harness can sweep policies without touching the simulator core.

The simulation harness (SIMULATION/, ~ 0.5 kLoC) implements the two-tier tick loop — per-card placement, NoC bandwidth accounting, pending-traffic queue — in pure NumPy; we deliberately avoid a hardware-cycle-accurate model because the scheduling decision is made entirely at admission, and the per-tick simulation only has to be accurate *enough* to faithfully expose the bandwidth ceiling and queue dynamics. The whole system is around 4.6 kLoC of Python and 1.8 kLoC of pytest tests, exercised through a Makefile that drives both the per-policy runs and the cross-scheduler comparison sweeps used in Section 7. The fingerprint extractor can also be invoked standalone (`python -m fingerprint.cli`) to produce synthetic fingerprints, which is what the steady-flat and pulse-train workloads in Section 7.1 are built from.

7 Evaluation

The evaluation is organised around four empirical claims. **RQ1** (Section 7.2): STPS consistently reduces end-to-end NoC congestion on a realistic mixed-fingerprint workload at a small throughput cost. **RQ2** (Section 7.3): macro-dispatch and phase-shift contribute along complementary axes, so neither stage alone explains the full result. **RQ3** (Section 7.4): STPS helps where topology is predictable and the cap is not in an extreme regime, and degrades gracefully outside that envelope. **RQ4** (Section 7.5): the delay cost is concentrated in the worst $\sim 1\%$ of per-task delays. In summary, among the four production baselines, STPS is the only scheduler that lowers NoC congestion in every cell tested at a 1–2% throughput cost; the two stages contribute along complementary axes and neither alone suffices; STPS helps where topology is predictable and the cap binds, and degrades gracefully elsewhere; and the entire cost is concentrated in the worst $\sim 1\%$ of per-task delays.

7.1 Trace-Driven Setup Exposes NoC Contention

The simulator models per-card NoC pressure. We use a discrete-event, trace-driven simulator of a multi-card CIM cluster (Section 5). Each card holds 256 cores wired by a 2D mesh NoC. The engine accounts for per-card instantaneous

demand, NoC bandwidth cap $BW_{\max}$, and a pending-traffic queue that absorbs over-budget injection rather than dropping the task; the queue is drained at the cap rate so no task is ever rejected and the cluster’s completion rate is always 1.0. All schedulers share the simulator and the same per-card placement primitive; they differ only in the admission-time dispatch and (for STPS) phase-shift logic.

Workloads mix synthetic extremes with real SNN traces. Each task draws a fingerprint from a *mixed* set: four synthetic templates (steady-flat, two pulse trains with periods $T=8$ and $T=16$, and a heavy-tail burst) plus three real SpikingJelly [2] traces (Spikformer [18]-CIFAR10, QKFormer-CIFAR10, SpikingResFormer-Tiny-ImageNet). Task arrivals follow either a Poisson process, a bursty arrival pattern that bunches admissions into onset windows, or a 1:1 mix of the two.

Trace-derived NoC injection is the modeled bottleneck. The simulator abstracts the cluster at the granularity our claims live at: per-card, per-tick NoC injection. Three properties make this sufficient to support the conclusions. First, the traffic timeline $E_{\tau}^{(t)}$ that drives every placement and phase-shift decision is *trace-derived*, not parametric: the three real fingerprints are extracted from actual SpikingJelly spike tensors via the offline DTDG pipeline (Section 5.2), so the burst shapes the scheduler reacts to are the ones the models actually produce. The four synthetic templates exist only to sweep extremes (perfectly flat, single-period, heavy-tail) that the real traces sample but do not isolate. Second, the contended resource we model — the NoC bandwidth cap and the pending-traffic queue draining at that cap — is the resource SNN inference on CIM actually serialises on: with weights resident in memory, inference is communication- not compute-bound, so an injection-rate model captures the first-order bottleneck while remaining agnostic to per-core compute timing that scheduling does not influence. Third, the cap itself is not a tuned knob: $BW_{\max}$ is fixed at the p75 of the per-card demand observed under an *uncapped* run, a workload-derived operating point at which every scheduler — ours included — sees congestion, rather than a value chosen to flatter STPS. We report the sensitivity of our conclusions to this choice across three cap-binding regimes in Section 7.4.

The methodology supports policy-level claims. Three boundaries follow from a simulation-first study. First, the bandwidth cap is set from the workload itself (p75 of uncapped demand) rather than calibrated against a specific silicon NoC, whose effective cap depends on routing, arbitration, and contention that an injection-rate model abstracts away; we bracket this uncertainty with the cap-binding sweep of Appendix A, whose binding, light, and non-binding regimes span the congestion levels a heavier (p50-like) or lighter (p90-like) cap would induce, and STPS’s qualitative ordering holds across all three. Second, the fingerprints are extracted from the same calibration traces against which the scheduler is evaluated; a deployment trace that drifts from the calibration

trace would erode Stage A’s topology score — an effect RQ3 characterises along the predictability axis but does not isolate as an explicit calibration-to-deployment mismatch. Third, the numbers are throughput, congestion, and delay in a discrete-event model, not wall-clock latency on hardware; closing that gap requires a cycle-accurate or silicon NoC study we leave to future work. We therefore read our results as evidence about scheduling *policy* under a faithful first-order NoC model, not as absolute performance predictions for a specific CIM part.

The default configuration fixes a congested operating point. Unless stated otherwise the cluster has $M=4$ cards, runs for 512 ticks, $BW_{\max}=9 \times 10^5$ (the p75 of the per-card demand observed under an uncapped run, so all schedulers see meaningful congestion), $D_{\max}=2$, $H=64$. We average each configuration over 5 seeds (21, 42, 99, 123, 2024) and report $\pm 95\%$ confidence half-widths where space permits; steady-state metrics skip the first and last 64 ticks of each run. The 16-card configuration scales the task count to 3200 to keep utilisation comparable.

Baselines are fingerprint-blind spatial schedulers. Round-robin (**RR**), best-fit on free cores (**BestFit**), dominant-resource fairness (**DRF**) [3], and power-of-two-choices (**P2C**) [11]. All four are spatial-only: they ignore fingerprints and inject tasks with zero offset. For the Stage-A ablation we isolate STPS’s macro-dispatch as the spatial-only variant STPS-SPATIAL; for the Stage-B ablation we additionally pair each baseline with the phase-shift module (+PS). The full STPS scheduler is the STPS-SPATIAL+PS pairing.

Metrics separate balance, congestion, throughput, and tail delay. We report (i) cross-card load coefficient of variation CV , Jain’s fairness index JFI [6] ($JFI = (\sum_m L_m)^2 / (M \sum_m L_m^2) \in (1/M, 1]$, 1 = perfectly balanced), the load-imbalance factor LIF ($LIF = \max_m L_m / \bar{L} \geq 1$, the busiest card’s load relative to the mean), and the max/min load ratio — four complementary views of across-card balance, with CV/JFI capturing dispersion and $LIF/\max\text{-min}$ the tail outlier; (ii) cluster *throughput* in tasks/tick, (iii) NoC *congestion ratio* (the fraction of demand pushed back by the cap) and the mean wait in the pending queue (*cong-wait*, ticks), and (iv) the p99 of per-task end-to-end delay, which we use to characterise the tail trade in Section 7.5. To separate Stage-B’s intrinsic offset from genuine queueing, we additionally report the *cold-start-excluded throughput*, $N_{\text{completed}} / (T_{\text{steps}} - \bar{T}_{\text{cold}})$, where $\bar{T}_{\text{cold}}$ is the per-run mean start offset committed by phase-shift.

7.2 RQ1: STPS Consistently Reduces NoC Congestion

Congestion falls in every cell. Table 1 reports STPS against the four baselines on the same workload at two cluster sizes. The primary result is in the congestion columns: STPS wins all six arrival/scale cells on both the congestion ratio and the mean queueing wait. No baseline wins on either metric in any cell. The size of the win is small in absolute terms

Table 1: End-to-end comparison on the mixed-fingerprint workload (5 seeds, mean values; bold = best in column within each cell). Among the four fingerprint-blind baselines, STPS is the *only* scheduler that wins on both congestion metrics — congestion ratio and mean queueing wait — in every {arrival, scale} cell, six out of six, at a 1–2 % throughput cost and a 6–20 tick increase in p99 tail delay. At 16 cards STPS is also the only policy whose worst-card max/min ratio stays under $13.5\times$ across all three arrivals, while every baseline spikes above $16\times$ — and P2C to $23.9\times$ — in at least one.

Scale	Arrival	Scheduler	CV ↓	JFI ↑	max/min ↓	tput ↑	cong. ratio ↓	cong. wait ↓	p99 ↓
4 cards	Poisson	RR	0.267	0.915	5.01	1.318	0.269	9.26	121.6
		BestFit	0.288	0.904	9.05	1.303	0.260	9.06	137.4
		DRF	0.288	0.904	9.05	1.303	0.260	9.06	137.4
		P2C	0.275	0.912	5.45	1.314	0.265	9.13	130.0
		STPS	0.293	0.903	5.46	1.292	0.247	8.42	141.4
	Bursty	RR	0.287	0.906	6.12	1.293	0.258	8.87	129.6
		BestFit	0.286	0.904	6.06	1.296	0.258	8.89	127.2
		DRF	0.286	0.904	6.06	1.296	0.258	8.89	127.2
		P2C	0.294	0.901	5.80	1.286	0.255	8.86	135.2
		STPS	0.307	0.896	6.23	1.269	0.239	8.14	143.4
	Mixed	RR	0.265	0.916	5.53	1.325	0.267	9.31	200.2
		BestFit	0.292	0.902	6.20	1.309	0.259	9.21	212.0
		DRF	0.292	0.902	6.20	1.309	0.259	9.21	212.0
		P2C	0.275	0.911	5.16	1.316	0.263	9.25	207.0
		STPS	0.300	0.898	6.24	1.297	0.245	8.56	215.4
16 cards	Poisson	RR	0.305	0.912	16.90	5.346	0.275	9.45	101.0
		BestFit	0.307	0.911	21.52	5.321	0.274	9.41	104.6
		DRF	0.307	0.911	21.52	5.321	0.274	9.41	104.6
		P2C	0.303	0.913	21.78	5.339	0.276	9.53	98.0
		STPS	0.317	0.906	13.42	5.274	0.256	8.61	107.2
	Bursty	RR	0.313	0.907	21.09	5.263	0.268	9.14	100.2
		BestFit	0.318	0.904	14.62	5.251	0.267	9.12	100.8
		DRF	0.318	0.904	14.62	5.251	0.267	9.12	100.8
		P2C	0.311	0.908	23.90	5.279	0.271	9.25	97.2
		STPS	0.325	0.901	12.09	5.212	0.252	8.45	105.4
	Mixed	RR	0.303	0.913	16.34	5.348	0.276	9.48	107.6
		BestFit	0.307	0.910	13.45	5.337	0.274	9.49	110.2
		DRF	0.307	0.910	13.45	5.337	0.274	9.49	110.2
		P2C	0.304	0.912	12.20	5.346	0.275	9.50	107.0
		STPS	0.316	0.906	12.80	5.289	0.257	8.62	114.2

(5–7 % on the ratio, 7–9 % on the wait) but its consistency distinguishes STPS from the baselines, which trade leadership across cells. The wins are also statistically separated: every congestion-ratio and queue-wait improvement in Table 1 exceeds the combined 95 % confidence half-widths of STPS and the best baseline in its cell. The tightest margin — 4-card bursty congestion ratio — is 0.016 against a combined half-width of 0.010; every other cell separates more cleanly, and the 16-card cells separate by an order of magnitude relative to their half-widths.

Congestion reduction costs 1–2 % throughput. STPS does not dominate the spatial metrics. Across the cells tested CV is 4–13 % higher than the best baseline and throughput is 1–2 % lower. Two distinct mechanisms produce this gap.

First, Stage B injects a non-zero start offset whenever the forecast peak would collide with the card’s existing schedule; this offset appears as the cold-start budget $\overline{T}_{\text{cold}} \approx 1.0$ tick under $D_{\text{max}} = 2$ and accounts for $\sim 0.2\%$ of the throughput delta (excluding it from the throughput denominator closes only 0.16–0.17 % of the 1–2 % gap). The remaining 0.8–1.8 % is contention avoidance: when phase-shift defers a task into a residual valley, the card injects strictly less per tick. The cluster trades a fraction of average throughput for lower instantaneous congestion. Second, STPS chooses cards by topology fit rather than by lowest-load match, so CV — which measures across-card variance of instantaneous served load — can be slightly higher than a pure load-balancer’s, even when the worst-card tail is smaller (see below).

Table 2: Throughput decomposition at 16 cards: cold-start-excluded throughput closes only 0.16–0.17 % of the 1.3–1.4 % gap to the best baseline. The remaining gap is contention-avoidance — the price STPS pays to keep the NoC under its cap.

Arrival	Scheduler	$\overline{T}_{\text{cold}}$	tput	tput _{excl. cs}
Poisson	best baseline (RR)	0.00	5.346	5.346
	STPS	1.02	5.274	5.282
Bursty	best baseline (P2C)	0.00	5.279	5.279
	STPS	1.01	5.212	5.221

STPS bounds tail skew at 16 cards. The max/min column captures a tail-skew effect that only emerges at scale. At 4 cards, where there are few placement choices, STPS is marginally behind the best load-balancer on worst-card skew (e.g. 5.46 vs RR’s 5.01 under Poisson), which is expected because STPS optimises congestion rather than average balance. At 16 cards the trend changes: STPS is the only scheduler whose worst-card ratio stays under $13.5\times$ across all three arrivals (13.4/12.1/12.8), whereas every baseline exceeds $16\times$ in at least one arrival and P2C reaches $23.9\times$ under bursty load. The capacity policies that look balanced at small scale develop a heavier tail exactly where the cluster has the most placement decisions to make; STPS’s fingerprint-aware dispatch keeps that tail bounded. This is consistent with the design intent of Section 5.3.1: the macro-dispatch score becomes more selective as more cards are available to score against, so the topology signal that fingerprint-aware dispatch carries scales with cluster size in a way that capacity-only policies cannot match.

The trade-off matches CIM contention. The RQ1 result supports a narrower claim than average load balancing: STPS targets the contention metric that dominates this CIM setting. In a substrate where the NoC is the dominant shared resource (Section 2.1), a 1–2 % throughput cost buys consistent NoC-pressure reduction and, at scale, bounds the worst-card ratio below $13.5\times$ rather than allowing the $24\times$ tail observed under P2C. This is the operating point STPS is designed to serve.

7.3 RQ2: Macro-Dispatch and Phase-Shift Are Complementary

STPS combines macro-card dispatch with phase-shift scheduling, so the evaluation must isolate the contribution of each stage. Table 3 holds the cap fixed and places the workload at a *lightly-congested* operating point ($BW_{\text{max}} = 9 \times 10^5$, $D_{\text{max}} = 2$, 400 tasks — regime C of Appendix A, where the cap clips only $\sim 8\text{--}10\%$ of demand rather than saturating it). We deliberately avoid the cap-saturated point: a hard-binding cap flattens every baseline to an artificially low CV (≈ 0.27) and

therefore masks how each stage actually reshapes load. The lightly-congested point lets the baselines express their natural across-card dispersion, so the stage decomposition is read against an honest reference. The table compares three variants for each policy family: the spatial policy alone (BASE), the same policy augmented with phase-shift (+PS), and STPS-SPATIAL — our macro-dispatch in isolation — with and without phase-shift. The STPS-SPATIAL+PS row is the full scheduler. Comparing each BASE row with its +PS counterpart isolates Stage B’s marginal effect; comparing the spatial-only rows isolates Stage A’s placement effect. Stage B is policy-agnostic by construction: it consumes the traffic timeline $E^{(t)}$ and the card forecast, scans up to D_{max} offsets, and commits the one whose forecast peak is lowest under the cap (Algorithm 1), so it can be attached to any spatial policy.

Phase-shift zeroes the mild congestion at $D_{\text{max}} = 2$. Table 3 reports the ablation. At this lightly-congested workpoint the baseline congestion ratio is already modest ($\sim 0.08\text{--}0.10$, queue wait 2.4–4.2 ticks); attaching phase-shift to RR, BestFit, DRF, or P2C drives it to exactly zero and empties the queue. The mechanism is direct: by deferring each task’s onset by at most $D_{\text{max}} = 2$ ticks the optimiser finds a slot whose superposition on the card forecast stays below BW_{max} , so p99 falls with it (e.g. RR Poisson 31.0 $\rightarrow$ 15.0). The cost is throughput: mean offsets stay near half a tick, but the cap still rejects the tasks that cannot find a sub-cap slot, and the steady-state window clips the deferred onsets, so throughput drops 17–23 % (RR Poisson 0.767 $\rightarrow$ 0.593). STPS-SPATIAL+PS (full STPS) is the exception that proves the point: it holds the congestion ratio at ~ 0.087 — essentially unchanged from STPS-SPATIAL’s 0.084 — because Stage A has already spread the load spatially, leaving almost nothing for Stage B to drain, and in return it pays *no* throughput (0.767, identical to its base) since its hotspot-split admission never triggers a cap rejection.

Phase-shift lowers CV when the cap does not clip. The central observation in Table 3 is that, unlike the cap-saturated regime, phase-shift here *lowers* CV for RR (Poisson 0.676 $\rightarrow$ 0.578), BestFit (0.647 $\rightarrow$ 0.574), and full STPS (0.659 $\rightarrow$ 0.615). The mechanism is mechanical and is exactly the sign-flip mapped in Appendix A: CV measures the across-card variance of instantaneous served load, and when the cap does *not* clip, de-synchronising bursts that previously peaked together lowers each card’s peak-to-mean ratio and so reduces across-card dispersion directly. This is the regime an operator should target, and it confirms that the CV inflation seen under a hard-binding cap is an artefact of the cap flattening the baselines to an artificially low ≈ 0.27 , not an intrinsic cost of phase-shift. The lone exception is P2C (0.717 $\rightarrow$ 0.832, 0.735 $\rightarrow$ 0.875): its power-of-two-choices base already decorrelates cards by instantaneous load, so adding temporal

Table 3: Stage decomposition at a *lightly-congested* workpoint (regime C of Appendix A): each spatial policy (BASE) paired with phase-shift (+PS), and STPS’s own macro-dispatch STPS-SPATIAL with phase-shift added; the full scheduler is STPS-SPATIAL+PS. 4 cards, mixed fingerprints, $BW_{\max} = 9 \times 10^5$, $D_{\max} = 2$, 400 tasks, 5 seeds. Baseline congestion is mild (~ 0.08 – 0.10); phase-shift drives it to zero on every spatial baseline. Because the cap no longer artificially flattens the baselines, phase-shift here *lowers* CV for RR/BestFit/STPS (the opposite of the cap-saturated regime); only the load-aware P2C base over-disperses. Full STPS holds a balanced metric vector — mild residual congestion (~ 0.087), the lowest CV among the STPS variants, and *no* throughput loss — where baseline+phase-shift zeroes congestion only by paying 17–23 % throughput.

Arrival	Policy	CV	throughput	cong. ratio	cong. wait	p99 delay	mean offset
Poisson	RR BASE	0.676	0.767	0.101	3.34	31.0	0.00
	RR +PS	0.578	0.593	0.000	0.00	15.0	0.43
	BestFit BASE	0.647	0.767	0.082	2.44	24.0	0.00
	BestFit +PS	0.574	0.638	0.000	0.00	15.6	0.46
	P2C BASE	0.717	0.766	0.098	3.52	29.0	0.00
	P2C +PS	0.832	0.610	0.000	0.00	16.8	0.63
	STPS-SPATIAL	0.659	0.767	0.084	2.48	25.0	0.00
	stps-spatial+ps	0.615	0.767	0.087	2.62	25.2	0.66
Bursty	RR BASE	0.707	0.758	0.103	3.98	32.8	0.00
	RR +PS	0.665	0.586	0.000	0.00	16.2	0.52
	BestFit BASE	0.683	0.763	0.087	3.18	30.2	0.00
	BestFit +PS	0.678	0.615	0.000	0.00	16.5	0.51
	P2C BASE	0.735	0.761	0.101	4.24	31.2	0.00
	P2C +PS	0.875	0.588	0.000	0.00	17.0	0.64
	STPS-SPATIAL	0.691	0.762	0.088	3.19	28.6	0.00
	stps-spatial+ps	0.662	0.760	0.088	3.17	29.8	0.66

de-synchronisation over-disperses an already-spread distribution. The sign of phase-shift’s CV effect is thus set by whether the cap binds and by how much spatial structure the base policy has already imposed, not by phase-shift in isolation.

The two stages occupy complementary roles. The two stages cover complementary failure modes, and the lightly-congested workpoint makes the division of labour legible. Phase-shift on a capacity baseline drives congestion to exactly zero but at a steep 17–23 % throughput cost, and for P2C it inflates CV; macro-dispatch alone (STPS-SPATIAL) holds CV and throughput at the baseline but leaves the mild congestion (~ 0.084) untouched. Only the pairing reaches a deployable compromise: full STPS posts the lowest CV among the STPS variants (0.615/0.662), keeps congestion mild (~ 0.087), and — crucially — loses *no* throughput, whereas every baseline+phase-shift row that zeroes congestion sacrifices nearly a fifth of it. Stage A primarily shapes the across-card distribution and protects throughput; Stage B reshapes each card’s timeline and, in this regime, pushes CV down rather than up. Their interaction prevents one stage from reintroducing the failure mode handled by the other. Table 3 should therefore be interpreted by considering each row’s full metric vector — congestion, CV, tail delay, and throughput together — not a single column: a policy that zeroes one number by giving up nearly a fifth of throughput is not the operating point STPS targets.

7.4 RQ3: STPS Helps in Predictable, Non-Extreme Regimes

STPS is not designed to dominate every metric in every operating regime. This section maps the boundary explicitly along three axes an operator controls: how predictable the workload’s topology is (does Stage A’s score carry signal?), how hard the bandwidth cap binds (is there congestion for Stage B to lever?), and how large the offset budget $D_{\max}$ is (how much tail may phase-shift spend?). The main result is that STPS’s two stages are *conditional* levers: each helps in the regime it was designed for and degrades gracefully outside it, and the relevant conditions are observable before deployment.

Stage A needs predictable fingerprints. Stage A consumes only two fingerprint quantities — the topological cohesion $\bar{K}_\tau$ and the burstiness β_τ — and emits a single scalar score. The question is: when does that score buy anything? We answer with two sweeps. The first varies cluster *utilisation* on a single workload mix; the second varies the *fingerprint composition* of the workload at fixed utilisation. Both are run on a 4-card cluster with 512 cores per card so that Stage A is exercised in isolation (without $BW_{\max}$ binding and without Stage B influence). All numbers below are 5-seed means; we report $\pm 95\%$ half-widths where they affect interpretation.

Utilisation determines whether topology matters. Figure 2 plots CV, so lower values indicate more even load across

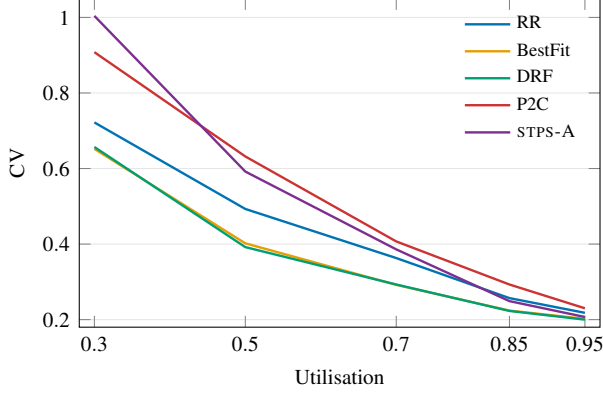


Figure 2: Stage A under a utilisation sweep (4 cards $\times$ 512 cores, Poisson, mixed fingerprints, 5 seeds). Each point reports CV, so lower is better. At low load BestFit/DRF dominate because empty-capacity signals overwhelm topology signals; from 0.85 onward Stage A converges into the leader band and at 0.95 all schedulers cluster in a 0.20–0.23 CV strip — the regime where physical saturation removes spatial degrees of freedom.

cards. The results reveal a clean regime structure. Below 70 % utilisation BestFit and DRF win because there is enough free capacity that almost any placement is acceptable, and the $\bar{K}_\tau$ -aware score adds noise rather than signal. Between 0.85 and 0.95 the four spatial policies converge into a tight 0.20–0.23 CV band: near saturation no policy has much spatial freedom to exploit. The informative window is the gap between these regimes, where Stage A is within 11 % of BestFit and ahead of RR/P2C — the operating range for which fingerprint-aware dispatch was designed.

Fingerprint predictability determines whether Stage A wins. Figure 3 fixes utilisation at 70 % and varies the workload from purely steady-flat to purely bursty fingerprints. Baselines do not read fingerprints, so their lines remain flat. Stage A’s line captures the role of predictability: at 100 % steady-flat it is the best scheduler in the figure (CV 0.290, -20.1 % vs RR, -0.9 % vs BestFit), matching the design intent of [17]-style topology-aware placement. As the bursty share grows the advantage erodes and, at 100 % bursty, Stage A becomes the worst of the five. This behaviour is expected. Stage A scores cards in constant time from the $H=64$ forecast horizon, but a $T=8$ pulse-train injects a fresh peak every eight ticks, so the horizon is saturated with peaks and the topology score cannot distinguish a quiet card from a busy one; capacity-based policies, which ignore the future altogether, become better defaults in the regime where the fingerprint no longer resolves a quiet placement. This is the first boundary: Stage A is a predictability lever, strongest when the fingerprint resolves the near future and neutral-to-negative when it cannot. It is exactly the bursty regime where Stage A

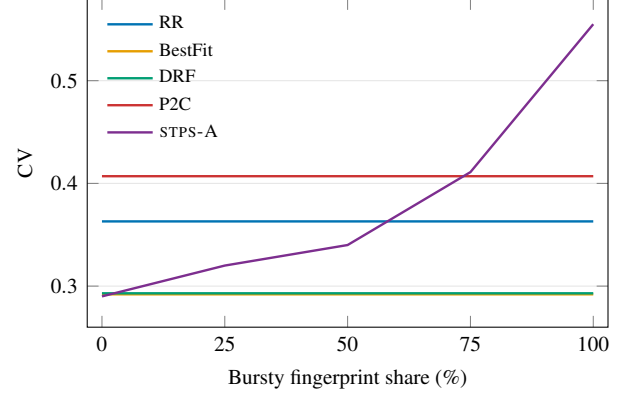


Figure 3: Stage A under a fingerprint-composition sweep (4 cards, Poisson, 70 % utilisation). Each point reports CV, so lower is better; the x-axis gives the bursty share of a steady-flat/bursty mix. Baselines ignore fingerprints and therefore form horizontal lines, while Stage A changes with workload topology. At 0 % bursty (100 % steady-flat), Stage A delivers the lowest CV of any scheduler — the design intent. The advantage erodes as the workload becomes more bursty, because $H=64$ forecast cannot resolve next-tick peaks.

weakens that Stage B, analysed next, takes over — the two stages cover complementary halves of the predictability axis.

Stage B works best before hard saturation. Phase-shift only relieves congestion where the cap binds. Sweeping the cap across three regimes — binding ($BW_{\max} = 9 \times 10^5$, 800 tasks; ~ 26 % baseline congestion), non-binding (5×10^6 , demand never reaches the cap), and lightly binding (9×10^5 , 400 tasks; ~ 9 %) — the congestion ratio drops to zero wherever there is congestion to remove (binding and light) and is undefined where there is none (non-binding). The load-balance metrics, by contrast, *reverse sign* with the regime: phase-shift lowers CV when the cap does not bind but raises it when the cap is already clipping peaks, because a binding cap is itself a crude flattener and de-synchronisation moves cards off that artificial uniformity. This sign-flip is universal — it holds for every baseline and for STPS-SPATIAL alike, so it is a property of the regime, not of STPS. The practical reading: STPS’s sweet-spot is a correctly- or over-provisioned cluster; pushed into heavy congestion, no scheduler improves load-balance, and the right response is provisioning, not a smarter policy. Appendix A gives the full per-policy grid.

A small offset budget captures most of the gain. A budget sweep $D_{\max} \in \{2, 4, 8, 16\}$ at the light cap (Table 4) shows the three groups of metrics pulling against each other. Congestion relief is binary: the moment phase-shift is enabled, every capacity baseline’s congestion ratio drops from ~ 0.09 to zero, and no larger budget improves on zero. Load-balance (CV,

Table 4: How each metric responds as the phase-shift budget $D_{\max}$ grows from 2 to 16 (light regime C, 4 cards, mixed fingerprints, 5 seeds). Congestion is a switch, not a knob; load-balance improves with diminishing returns and saturates near $D_{\max}=8$; tail delay and throughput cost grow monotonically with the offset budget. The three columns pull in opposite directions, which is what fixes a small default.

Metric	Response to $D_{\max} \uparrow$	Detail
Cong. ratio (non-STPS)	flat at 0 from $D_{\max}=2$	switch, not knob
Cong. ratio (STPS)	slow $\downarrow$	$0.087 \rightarrow 0.079$
CV / JFI / LIF	$\downarrow$, saturating	knee near $D_{\max}=8$
Max/min	$\downarrow$	$8-13 \rightarrow 3-6$
p99 / mean offset	$\uparrow$ monotone	offset $0.5 \rightarrow 4$ ticks
Throughput	partial recovery	stays below baseline

JFI, LIF) improves with diminishing returns and saturates near $D_{\max}=8$ — the RR CV bottoms at 0.44 for $D_{\max}=8$ then rebounds at 16. But tail delay moves the opposite way: the mean injected offset grows from ~ 0.5 to ~ 4 ticks across the sweep, dragging p99 up monotonically and clawing back the throughput that the cap rejections cost. The budget that buys the congestion switch and most of the load-balance gain while paying the least tail and throughput is the smallest one. We adopt $D_{\max}=2$ as the default: it already zeroes congestion, recovers the bulk of the max/min compression ($8-13\times \rightarrow 3-6\times$), and holds the mean offset under one tick. On full STPS the case is sharper still — a larger budget yields no congestion or balance gain (Stage A has already consumed the spatial slack) and only inflates p99, so $D_{\max}=2$ is strictly preferred. The cap-binding sweep (Appendix A) explains which of these signals would persist or invert under a heavier cap, even at this attenuated budget.

7.5 RQ4: Tail Delay Is Confined to the Worst 1%

Averages flatter cluster schedulers. A scheduler that improves the median can still fail an SLA at the tail, and the tail is exactly where STPS’s trade-off lives. Figure 4 plots $\Pr[\text{delay} > d]$ for both the 4-card and 16-card clusters under both arrival modes; we show the two extreme arrival processes, omitting the Mixed panels because, as a 1:1 blend of Poisson and bursty, their survival curves lie between the two extremes and reveal no additional shape. The four panels tell a consistent story: up to roughly the 90th percentile every scheduler tracks every other within a few ticks; the curves only fan out in the upper-right corner. By construction, the tasks that populate that corner are the long-tail tasks — the same bursty, high- β_τ population that defines p99 in the first place — and STPS’s slower-decaying curve sits to the right of the baselines precisely on those tasks. At p99 on 16 cards STPS trails the best baseline by 6–9 ticks (e.g. 107 vs 98

under Poisson, 105 vs 97 under bursty); the 4-card panels show the same shape at a larger absolute tail price ($\sim 15-20$ ticks). Crucially, the curves are indistinguishable on the body: STPS does not slow median or p90 tasks; it spends its delay budget exclusively on the worst $\sim 1\%$.

Bursty-task offsets create the tail. Phase-shift commits each task to a non-zero start offset whenever the chosen card’s forecast peak would otherwise collide with the task’s burst. Over a long run, the offset accumulates on the bursty tasks — the ones whose β_τ is large — because they are the ones whose peaks need shifting. The same tasks dominate the worst-card load in the baselines; STPS finishes them 5–10% later and uses the slack to keep the worst card under control. In exchange, the cluster as a whole sees the gains already documented: congestion ratio $-5-7\%$, mean queue wait $-7-9\%$, 16-card max/min $14-24\times \rightarrow 12-13\times$. The same offsets that lengthen the worst 1% of tasks are what flatten the cards underneath them.

The trade is favourable for congestion-bound serving.

An operator with deadlines tighter than the 6–12 tick p99 gap should run STPS with a smaller $D_{\max}$; the offset-budget sweep (Section 7.4) shows that even $D_{\max}=2$ already keeps the static-baseline tail price under 3 ticks. For workloads whose dominant operational concern is cluster-wide congestion and worst-card load — the typical multi-tenant CIM serving setting we target — the trade is favourable: STPS sacrifices the worst 1% of tasks to keep the worst card under control, and the worst card is what saturates first.

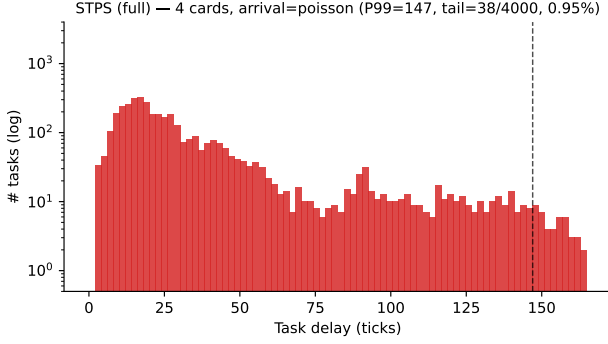
8 Conclusion

STPS treats SNN scheduling as a problem of admission-time decisions over a small, physical workload fingerprint, rather than as a per-spike optimisation or a capacity-only bin-packing. Three DTDG-derived quantities suffice to drive two closed-form stages — macro-dispatch and phase-shift — whose combined effect is to keep NoC pressure flat where capacity-based schedulers let bursts collide. In simulation, this trade buys consistent congestion relief at a modest throughput cost, with each stage contributing along a complementary axis. The fingerprint abstraction has room to extend: a fourth quantity, the per-population hotspot indicator, is already extracted but not yet acted on, and the phase-shift kernel assumes a stationary forecast horizon. A scheduler that splits hotspot populations across cards under tail-imbalance pressure, or that re-evaluates the offset choice when the forecast drifts, is a natural next step.

Acknowledgments

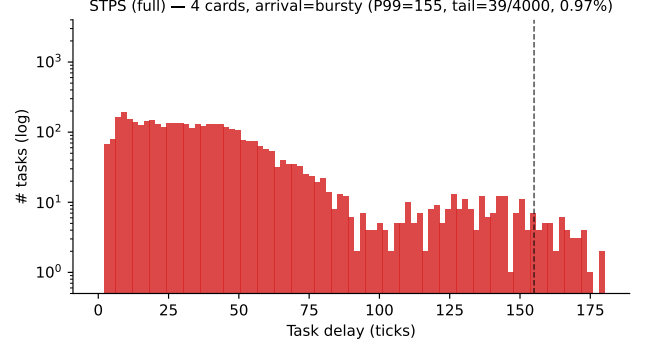
Anonymised for review.

Long-tail delay distribution — STPS, 4 cards, arrival=poisson, 5 seeds pooled



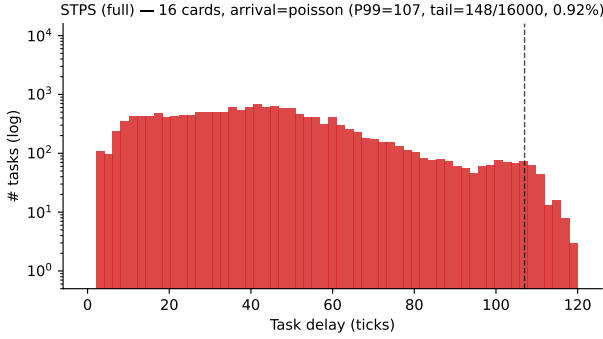
(a) 4 cards, Poisson

Long-tail delay distribution — STPS, 4 cards, arrival=bursty, 5 seeds pooled



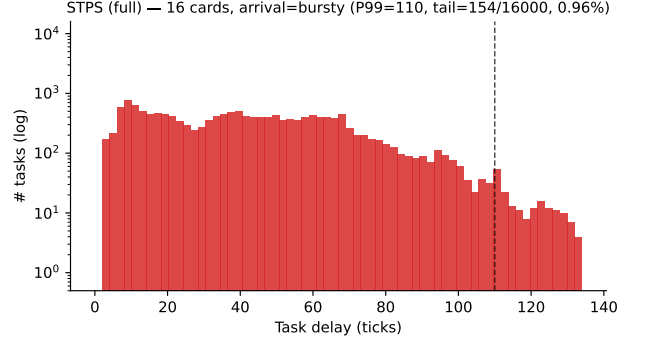
(b) 4 cards, Bursty

Long-tail delay distribution — STPS, 16 cards, arrival=poisson, 5 seeds pooled



(c) 16 cards, Poisson

Long-tail delay distribution — STPS, 16 cards, arrival=bursty, 5 seeds pooled



(d) 16 cards, Bursty

Figure 4: Delay survival functions across both cluster sizes and both arrival modes (5 seeds pooled, log-log). All schedulers track each other up to roughly the 90th percentile; STPS’s curve only departs from the baselines above $\Pr[\text{delay} > d] \approx 10^{-2}$. The tasks living in that upper-right corner — the ones STPS finishes later — are the same bursty, high- β_r population that drives p99: STPS pays its tail price exclusively on the long-tail tasks, and pays it to buy the cluster-wide load-balancing and congestion gains documented in Table 1.

Availability

The simulator, the offline DTDG fingerprint extractor, and the experimental harness used to produce every table and figure in this paper will be released under a permissive open-source licence at publication.

A Cross-Regime Cap-Binding Study

Section 7.4 summarises how phase-shift’s effect depends on whether the bandwidth cap binds; this appendix gives the full three-regime sweep behind that summary.

The three workpoints. The sweep re-runs the full Stage-B ablation under three operating regimes that vary independently along the cap-binding axis while keeping every other knob fixed (4 cards, mixed fingerprints, $H = 64$, $D_{\max} = 16$,

5 seeds; $D_{\max} = 16$ amplifies the signal that $D_{\max} = 2$ shows in attenuated form). Regime A (binding, $BW_{\max} = 9 \times 10^5$, 800 tasks) places demand at the p75 of an uncapped run, so the cap clips roughly 26 % of the demanded injection and forces a deep pending-traffic queue. Regime B (non-binding, $BW_{\max} = 5 \times 10^6$, 800 tasks) raises the cap by $5.5\times$ so demand never reaches it; baselines never queue. Regime C (lightly binding, $BW_{\max} = 9 \times 10^5$, 400 tasks) halves the task count, leaving the same cap but only $\sim 9\%$ baseline congestion. The three regimes are designed to span the operationally relevant range: a CIM cluster that is over-provisioned (B), correctly provisioned (C), or under-provisioned (A).

CV reverses sign across regimes, congestion does not. Tables 6 and 7 report the three-regime sweep. The CV column tells a clean story: every cell of regime A is negative (phase-shift *raises* CV); every cell of regimes B and C is pos-

Table 5: Cross-regime cap-binding profile (5 seeds, mean values). The three workpoints differ along a single axis — how heavily the bandwidth cap binds the baseline schedulers. The *offset-infeasible rate* counts tasks for which phase-shift finds no offset in $[0, D_{\max}]$ that keeps the forecast peak below the cap; such a task is still admitted at offset 0 (no task is ever dropped, cf. Section 7.1), but phase-shift cannot relieve its contribution to congestion.

	A: binding 9×10^5 , 800	B: non-binding 5×10^6 , 800	C: light 9×10^5 , 400
Baseline cong. ratio	~ 0.26	0.00	~ 0.09
Baseline CV	0.27–0.29	0.66–0.78	0.65–0.74
Baseline p99 (ticks)	120–137	15–16	24–33
Offset-infeasible (+PS)	0.18–0.22	0.00	0.14–0.16

Table 6: Cross-regime ΔCV under +PS (5 seeds, $(off - on)/off \times 100$; positive = phase-shift reduces CV). Every cell of the binding regime is negative; every cell of the non-binding and light regimes is positive. The direction of phase-shift’s effect on CV is determined entirely by whether the cap is binding, independent of arrival mode, baseline policy, or even the spatial policy that phase-shift is attached to.

Arrival	Policy	A: binding	B: non-bind.	C: light
Poisson	RR	−33.7	+25.9	+28.5
	BestFit/DRF	−11.8	+12.7	+25.8
	P2C	−43.9	+27.2	+8.4
	STPS-SPATIAL	−49.7	+19.8	+8.8
Bursty	RR	−34.1	+22.0	+17.8
	BestFit/DRF	−11.8	+12.5	+19.1
	P2C	−45.2	+25.1	+2.8
	STPS-SPATIAL	−47.3	+15.4	+6.7

itive (phase-shift *lowers* it). The sign of phase-shift’s effect on CV is determined entirely by cap binding — not by arrival mode, not by the baseline policy, not even by whether the baseline is a capacity scheduler or STPS-SPATIAL. The congestion ratio, by contrast, moves monotonically: $\downarrow$ in A and C, undefined (already 0) in B. Phase-shift is therefore a congestion lever in the regime where there is congestion to lever and a CV reducer in the regime where there is not; the underlying mechanism — de-synchronising bursts in time — is the same, but the metric that registers the benefit changes with the workpoint.

The mechanism, broken down. In regime A the cap is already flattening peaks by clipping them; baseline cards therefore look uniform on the served-load axis ($CV \approx 0.28$), and phase-shift’s de-synchronisation moves them off that artificial uniformity, raising CV while emptying the queue. In regime B no demand is clipped; phase-shift spreads aligned bursts in time, lowering the peak-to-mean ratio on every card and reduc-

Table 7: Cross-regime direction summary for the remaining Stage-B metrics. Phase-shift relieves congestion only where the cap binds (A, C), and only relieves p99 where the baseline queue already dominates p99 (A). The throughput cost is determined by the offset budget’s ratio to the steady-state window, not by the cap.

Metric direction	A	B	C
CV	$\uparrow$	$\downarrow$	$\downarrow$
JFI	$\downarrow$	$\uparrow$	$\uparrow$
LIF	$\uparrow$	$\downarrow$	$\downarrow$
Max/min (non-STPS)	$\downarrow$	$\downarrow$	$\downarrow$
Cong. ratio	$\downarrow (.26 \rightarrow 0)$	—	$\downarrow (.09 \rightarrow 0)$
Throughput cost	3–11%	3–10%	14–16%
p99 (non-STPS)	−75/−78	+444/+625	−9/−48
p99 (STPS+ps)	+68/+82	+445/+534	+20

p99 rows: low/high % across seeds and arrival modes.

ing CV directly. Regime C is the intermediate case: enough congestion to give phase-shift something to lever (cong. ratio drops from ~ 0.09 to ~ 0), not enough to invert the CV signal (still -2.8% to $+28.5\%$). Notably, the regime B and C numbers include STPS-SPATIAL+ps as a row, which lands at $+19.8\%$ and $+8.8\%$ under Poisson; the Pareto pathology of regime A on STPS+phase (Section 7.5) does not transfer to the regimes where the cap does not bind.

Extreme congestion is hostile to every scheduler, not just STPS. Read horizontally rather than per-policy, Table 6 delivers a sharper message: in regime A *every cell is negative*, across both arrival modes and across all four spatial policies — RR (−33.7/−34.1 %), BestFit/DRF (−11.8 % twice), P2C (−43.9/−45.2 %), and STPS-SPATIAL (−49.7/−47.3 %). The load-balance regression is therefore not an STPS-specific failing; it is a property of the regime. The same horizontal reading of Table 7 shows the rest of the load-balance panel moving in lockstep — JFI $\downarrow$ and LIF $\uparrow$ for everyone in column A — while regimes B and C invert every sign. The implication for the operator is direct: STPS’s operating sweet-spot is the non-extreme regimes (B, C), where phase-shift’s effect on CV flips positive across *every* policy and Stage A’s placement gains compound with Stage B’s de-synchronisation. Once the cluster is pushed deep into regime A, no scheduler in this study — baseline or STPS — improves load-balance under phase-shift; the cap itself has become the flattener, and any further temporal de-synchronisation only re-introduces variance the cap had clipped away. The correct response to regime A is therefore not a smarter scheduler but a provisioning or admission-control decision that moves the cluster out of it; the next paragraph quantifies which of the two interventions is cleaner.

An asymmetry between two ways to relieve the cap. Regime C halves the demand by halving the task count; regime B raises the cap by 5.5 $\times$. Both are operationally meaningful ways for a cluster operator to "remove congestion". The offset-infeasible column of Table 5 shows they are not equivalent: halving demand only drops the rate from ~ 0.20 to ~ 0.15 , while raising the cap drives it to zero. Phase-shift's downstream effect on CV is consistent with this asymmetry — regime B's improvement (12.5–27.2 %) is larger and more uniform than C's (2.8–28.5 % with wider spread). For an operator deciding whether to provision more bandwidth or to throttle admissions, the regime sweep argues that bandwidth provisioning is the cleaner intervention for taking advantage of phase-shift.

References

- [1] Mike Davies, Narayan Srinivasa, Tsung-Han Lin, Gautham Chinya, Yongqiang Cao, Sri Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Imam, Shweta Jain, et al. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1):82–99, 2018.
- [2] Wei Fang, Yanqi Chen, Jianhao Ding, Zhao Fei Yu, Timothée Masquelier, Ding Chen, Liwei Huang, Huihui Zhou, Guoqi Li, and Yonghong Tian. Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence. *Science Advances*, 9(40):eadi1480, 2023.
- [3] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *8th USENIX symposium on networked systems design and implementation (NSDI 11)*, 2011.
- [4] Robert Grandl, Ganesh Ananthanarayanan, Srikanth Kandula, Sriram Rao, and Aditya Akella. Multi-resource packing for cluster schedulers. *ACM SIGCOMM Computer Communication Review*, 44(4):455–466, 2014.
- [5] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving DNNs like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 443–462, 2020.
- [6] Rajendra K Jain, Dah-Ming W Chiu, William R Hawe, et al. A quantitative measure of fairness and discrimination. *Eastern Research Laboratory, Digital Equipment Corporation, Hudson, MA*, 21(1):2022–2023, 1984.
- [7] Sangmin Lee, Rina Panigrahy, Vijayan Prabhakaran, Venugopalan Ramasubramanian, Kunal Talwar, Lincoln Uyeda, and Udi Wieder. Validating heuristics for virtual machines consolidation. *Microsoft Research, MSR-TR-2011-9*, pages 1–14, 2011.
- [8] De Ma, Xiaofei Jin, Shichun Sun, Yitao Li, Xundong Wu, Youneng Hu, Fangchao Yang, Huajin Tang, Xiaolei Zhu, Peng Lin, and Gang Pan. Darwin3: A large-scale neuromorphic chip with a novel isa and on-chip learning. *National Science Review*, 2024.
- [9] Christian Mayr, Sebastian Höppner, and Steve Furber. Spinnaker 2: A 10 million core processor system for brain simulation and machine learning. *arXiv preprint arXiv:1911.02385*, 2019.
- [10] Paul A Merolla, John V Arthur, Rodrigo Alvarez-Icaza, Andrew S Cassidy, Jun Sawada, Filipp Akopyan, Bryan L Jackson, Nabil Imam, Chen Guo, Yutaka Nakamura, et al. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, 345(6197):668–673, 2014.
- [11] Jayashree Mohan, Amar Phanishayee, Janardhan Kulkarni, and Vijay Chidambaram. Looking beyond {GPUs} for {DNN} scheduling on {Multi-Tenant} clusters. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 579–596, 2022.
- [12] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. {Heterogeneity-Aware} cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 481–498, 2020.
- [13] Garrick Orchard, E. Paxon Frady, Daniel Ben Dayan Rubin, Sophia Sanborn, Sumit Bam Shrestha, Friedrich T. Sommer, and Mike Davies. Efficient neuromorphic signal processing with loihi 2. In *IEEE Workshop on Signal Processing Systems (SiPS)*, 2021.
- [14] Rina Panigrahy, Kunal Talwar, Lincoln Uyeda, and Udi Wieder. Heuristics for vector bin packing. *research.microsoft.com*, 2011.
- [15] Sultan Mahmud Sajal, Luke Marshall, Beibin Li, Shandan Zhou, Abhisek Pan, Konstantina Mellou, Deepak Narayanan, Timothy Zhu, David Dion, Thomas Moscibroda, and Ishai Menache. Kerveros: Efficient and scalable cloud admission control. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 227–245, Boston, MA, July 2023. USENIX Association.
- [16] Qizhen Weng, Lingyun Yang, Yinghao Yu, Wei Wang, Xiaochuan Tang, Guodong Yang, and Liping Zhang. Beware of fragmentation: Scheduling {GPU-Sharing}

workloads with fragmentation gradient descent. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 995–1008, 2023.

- [17] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, et al. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 595–610, 2018.
- [18] Zhaokun Zhou, Yuesheng Zhu, Chao He, Yaowei Wang, Shuicheng Yan, Yonghong Tian, and Li Yuan. Spikformer: When spiking neural network meets transformer. In *International Conference on Learning Representations (ICLR)*, 2023.